Projet Bases de Données Avancées

Bases de Données Avancées — 2026-2027 · Projet de fin de module

Le projet se réalise en groupes de 2 et se rend le **29 novembre 2026 à 23h59** sur la page Moodle prévue à cet effet.

C'est la base de données qui est évaluée, pas l'application. Vous pouvez développer l'application comme vous le souhaitez, y compris en vibe coding avec une IA. En revanche, chaque membre du groupe doit être capable d'expliquer et de modifier en direct le schéma, les requêtes, les objets programmés et la politique de sécurité de la base. Cette compréhension est vérifiée à chaque séance de suivi.

Choix du sujet

Le sujet est libre : chaque groupe choisit un domaine, conçoit ses propres tables et relations, et fait valider son sujet par l'enseignant en séance 1.

Un sujet est recevable s'il réunit les éléments suivants, sans lesquels les notions du cours ne pourraient pas être mises en œuvre :

- **au moins 5 tables** liées entre elles, dont une association plusieurs-à-plusieurs ;
- **au moins 2 profils d'utilisateurs** aux droits différents ;
- **des données à cloisonner** : certains utilisateurs ne doivent voir ou modifier qu'une partie des lignes d'une table, ce qui justifie la RLS ;
- **au moins une règle métier** qu'une simple contrainte ne peut pas exprimer, ce qui justifie un trigger ou une procédure ;
- **au moins un indicateur à calculer**, ce qui justifie une vue matérialisée.

En séance 1, chaque groupe présente une **fiche sujet d'une page** : domaine, acteurs et rôles, entités principales, règles de cloisonnement et au moins 3 règles métier. Dans la suite de ce document, les exemples sont donnés à titre d'illustration et sont à transposer à votre sujet.

Objectifs et contraintes techniques

Votre objectif est de livrer une base de données SQL complète, sécurisée et rejouable, pilotée par une application qui l'utilise réellement.

1. Concevoir le modèle de données (MCD puis MLD) de votre sujet.
2. Créer le schéma et un jeu de données réaliste.
3. Implémenter les objets avancés vus en cours : requêtes complexes, vues et vues matérialisées, fonctions, procédures, triggers, rôles et privilèges, Row-Level Security.
4. Développer une application reliée à la base, avec une couche DAO qui centralise tous les accès.

Contraintes :

- **Langage et framework de l'application libres** (Java, Python, JavaScript/TypeScript, PHP, C#...). Un ORM est autorisé pour le mapping, mais les requêtes complexes, appels de procédures et fonctions doivent apparaître explicitement en SQL dans les DAO.

- **Scripts rejouables** : la base doit pouvoir être recréée de zéro en exécutant les scripts SQL dans l'ordre, sans erreur. Organisation attendue dans le dépôt :
  - `sql/01_schema.sql` — tables, contraintes, index
  - `sql/02_data.sql` — jeu de données
  - `sql/03_views.sql` — vues et vues matérialisées
  - `sql/04_functions_procedures.sql`
  - `sql/05_triggers.sql`
  - `sql/06_security.sql` — rôles, privilèges, politiques RLS
  - `sql/07_tests.sql` — scénarios de démonstration et de test
- **Volume de données minimal** : quelques centaines de lignes dans la table principale, pour que les EXPLAIN ANALYZE et les vues matérialisées aient un sens. Un script de génération est autorisé et recommandé.
- **L'application ne se connecte jamais avec le super-utilisateur** ni avec le propriétaire des tables : elle utilise les rôles applicatifs définis dans `06_security.sql`.

Exigences sur la base de données

Un exemple de chaque notion suffit, mais c'est à vous d'en trouver l'utilité dans votre sujet. Aucun usage n'est imposé : vous identifiez où chaque notion répond à un vrai besoin, vous la mettez en œuvre et vous la mettez en valeur dans le rapport et dans la vidéo. La quantité n'est pas notée ; un objet ajouté sans utilité réelle ne rapporte pas de points.

Notion	Minimum attendu
Modélisation	5 tables en 3FN, contraintes complètes, 1 index justifié
Requête complexe	1 requête combinant au moins 3 techniques
Vue	1
Vue matérialisée	1, avec stratégie de rafraîchissement
Fonction	1
Procédure	1, transactionnelle avec gestion d'erreurs
Trigger	1
Rôles et privilèges	2 rôles applicatifs au moindre privilège
Row-Level Security	1 table protégée

Exigences sur l'application

L'application sert de vitrine à la base : elle est peu notée, mais elle doit montrer chaque objet de la base en action.

- **Modèles** : une classe ou structure par entité métier principale de votre modèle.
- **Couche DAO** : une interface (ou classe abstraite) par entité et une implémentation SQL. Aucune requête SQL en dehors des DAO.
- **Requêtes paramétrées obligatoires** (requêtes préparées) ; toute concaténation de valeurs dans une requête SQL est pénalisée.
- **Appels visibles** aux objets avancés : les DAO appellent les procédures (`CALL`), les fonctions, lisent les vues et vues matérialisées, et au moins une requête complexe.

- **Connexion par rôle** : l'application change de rôle ou positionne la variable de session selon l'utilisateur connecté, de sorte que la RLS filtre réellement les données affichées.
- **Gestion des erreurs** : une erreur levée par un trigger ou une procédure (stock insuffisant, droit refusé) est affichée proprement à l'utilisateur.
- **Écrans minimum** : connexion (choix d'un utilisateur de test suffit), au moins un écran par rôle applicatif, un écran qui déclenche une procédure, un tableau de bord alimenté par les vues matérialisées.
- **README** : installation, création de la base, lancement, comptes de test par rôle.

Si vous développez l'application avec une IA, signalez-le dans le README. Vous restez responsables de chaque requête présente dans les DAO et pourrez être interrogés dessus.

Déroulement

Le projet s'étale sur 4 séances de suivi ; chacune valide un livrable d'étape. La vidéo de rendu fait office de soutenance. Un livrable non validé en séance peut l'être à la suivante, mais compte alors pour moitié dans la note de suivi.

Séance	Thème	Livrable à présenter
1	Conception	Groupe constitué, fiche sujet validée, règles de gestion, MCD et MLD v1
2	Schéma et données	`01_schema.sql` et `02_data.sql` exécutables, première version de la requête complexe
3	Logique en base	`03` à `05` : vue, vue matérialisée, fonction, procédure, trigger ; requête complexe et `EXPLAIN ANALYZE`
4	Sécurité et application	`06_security.sql` et `07_tests.sql` ; application connectée à la base via les DAO

À chaque séance, l'enseignant interroge un membre du groupe choisi au hasard sur le livrable présenté.

Livrables finaux

Tout est déposé sur Moodle ou sur Github avant le **29 novembre 2026 à 23h59** ; un retard entraîne une pénalité d'un point par jour.

1. **Rapport (PDF)** contenant :
   - la présentation du sujet et de ses acteurs, les règles de gestion, le MCD et le MLD commentés ;
   - la liste des objets avancés, avec pour chacun son rôle et le besoin métier auquel il répond ;
   - la répartition du travail dans le groupe.
2. **Lien vers le dépôt GitHub** contenant les scripts SQL `01` à `07`, le code de l'application et le README. Le dépôt doit être accessible à l'enseignant.
3. **Vidéo de démonstration (10 minutes maximum)** : Présentation de votre projet, demo interaction entre base et application, utilisations des différents éléments SQL demandés

Grille d'évaluation

La base de données et sa compréhension pèsent 16 points sur 20 ; l'application n'en pèse que 2, au titre de son intégration avec la base. Dans chaque critère, la pertinence de l'objet pour le sujet et sa mise en valeur comptent plus que sa complexité.

Critère	Attendu pour la note maximale	Barème
Suivi des séances	Les 4 livrables d'étape validés à temps ; l'étudiant interrogé répond correctement	4 (1 par séance)
Modélisation et schéma	MCD/MLD cohérents avec les règles de gestion, 3FN, contraintes présentes et justifiées, types adaptés, sujet recevable	2,5
Requête complexe et optimisation	Requête utile et correcte combinant au moins 3 techniques ; index justifié par un `EXPLAIN ANALYZE` commenté	2
Vue et vue matérialisée	Utilité démontrée dans le sujet ; `security_invoker` maîtrisé si besoin ; rafraîchissement et protection de la vue matérialisée expliqués	1,5
Fonction, procédure, trigger	Chaque objet répond à un besoin réel du sujet ; procédure atomique avec erreurs explicites ; choix `BEFORE` / `AFTER` et `ROW` / `STATEMENT` justifiés	2,5
Rôles et privilèges	Propriétaire non connectable, `REVOKE` sur `PUBLIC`, 2 rôles au moindre privilège, matrice des droits exacte	1,5
Row-Level Security	Cloisonnement justifié par le sujet, RLS activée et forcée, `USING` / `WITH CHECK` corrects, identification de l'utilisateur maîtrisée, test probant	2
Intégration application et DAO	DAO par entité, requêtes paramétrées, appels à la procédure, à la fonction et aux vues, connexion par rôle effective, erreurs de la base affichées proprement	2
Rapport, vidéo et dépôt	Livrables complets ; chaque notion mise en valeur avec le besoin auquel elle répond ; scripts rejouables de zéro ; README suffisant	2
Total		**20**

Pénalités, appliquées après le total :

- scripts SQL qui ne s'exécutent pas de zéro dans l'ordre : −2 ;
- valeurs concaténées dans une requête SQL (risque d'injection) : −1 ;
- application connectée avec `root` ou avec le propriétaire des tables : −1 ;
- retard de rendu : −1 par jour entamé.